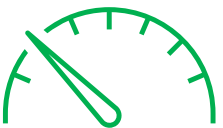


Métricas de Calidad



Docente: Ing. Diego Gamboa Mgtr.

dlgamboa1@espe.edu.ec



Contenidos

- Introducción a las métricas de calidad del software.
- Recopilación de métricas de Calidad.
- Estándares de métricas: ISO/IEC 25022
- Métricas de procesos de software

Contenidos

- Métricas del producto.
- Métricas y prácticas ágiles.
- Desafíos en la implementación de métricas de software.



01

Introducción a las métricas de calidad del software

Conceptos de métricas de calidad del software



Herramientas para evaluar
rendimiento, mantenibilidad y
eficiencia del software

Indicadores clave que apoyan
la mejora continua en procesos
de desarrollo



Métodos cuantitativos para
medir calidad técnica y
satisfacción del usuario



Clasificación de las métricas de calidad del software



Métricas de producto
enfocadas en características
como funcionalidad,
rendimiento, y mantenimiento

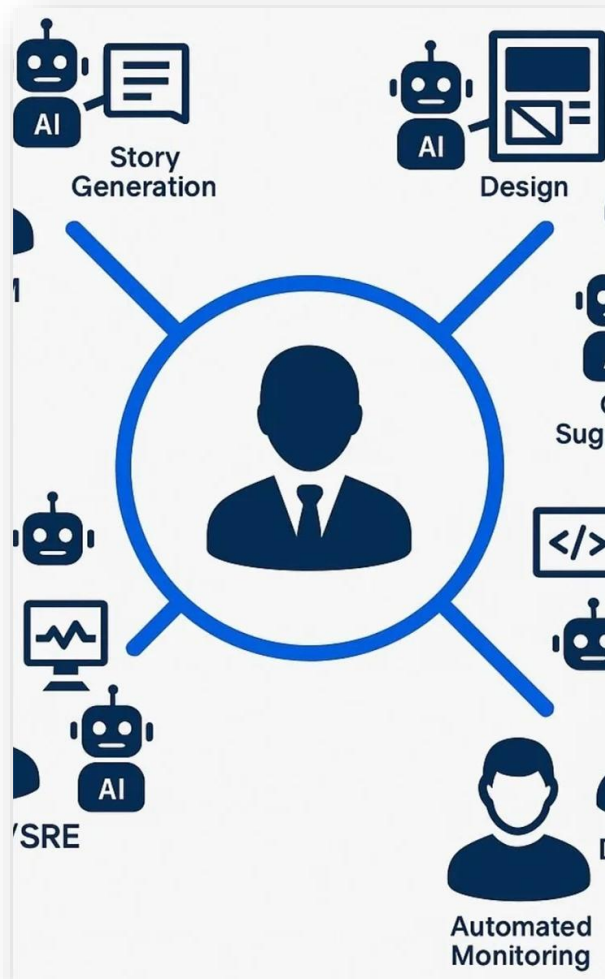
Métricas de proceso evaluando
eficiencia y calidad en el ciclo
de desarrollo



Métricas de proyecto
analizando costos, tiempos, y
recursos utilizados



El papel de las métricas en el desarrollo de software



Toma de decisiones

Las métricas proporcionan datos objetivos para priorizar tareas, asignar recursos y enfocarse en áreas críticas de mejora.

Optimización del ciclo de vida

Ayudan a evaluar las fases de desarrollo, pruebas y mantenimiento, asegurando eficiencia y cumplimiento de plazos.

Identificación temprana de riesgos

Las métricas revelan problemas potenciales como deuda técnica y desafíos de rendimiento antes de que afecten al producto final.

Beneficios del uso de métricas de calidad

Mejora continua

Facilitan ajustes iterativos para garantizar un proceso de desarrollo más eficiente y mejores resultados finales.



Reducción de errores

Al detectar problemas de diseño y calidad, ayudan a minimizar defectos y mejorar la estabilidad del software.

Aumento de satisfacción

Garantizan que el producto cumpla con las expectativas del usuario mediante la entrega de valor y funcionalidad consistente.



02

Recopilación de métricas de calidad

Técnicas de Recopilación de Datos

Encuestas

Utilización de cuestionarios estructurados para recopilar datos cualitativos sobre la percepción de calidad del software entre usuarios y desarrolladores.

Herramientas automatizadas

Empleo de programas especializados para extraer métricas de rendimiento y defectos desde el código en tiempo real.

Pruebas de usuario

Realización de pruebas controladas con usuarios finales para evaluar la usabilidad y funcionalidad básica de una aplicación.

Registro de datos

Análisis de logs del sistema para identificar puntos de fallo recurrentes y patrones de uso.

Herramientas de Métricas de Calidad de Software

Métricas de Código Fuente:

Herramienta	Descripción	Soporte de lenguajes
SonarQube	Plataforma líder para análisis estático de código. Proporciona métricas de calidad, bugs, vulnerabilidades y deuda técnica.	Java, C#, JavaScript, Python, PHP, etc. (más de 30 lenguajes)
SonarLint	Versión ligera de SonarQube integrada en IDE (VS Code, IntelliJ, etc.)	Similar a SonarQube
CodeClimate	Análisis automatizado de calidad, enfocado en mantenibilidad y seguridad.	Ruby, JavaScript, Python, Go, etc.
ESLint + plugins (como eslint-plugin-complexity)	Para métricas específicas en JavaScript/TypeScript (ej. complejidad, duplicación)	JS/TS
Pylint / Flake8 / Radon	Herramientas para métricas en Python (complejidad ciclomática, LOC, etc.)	Python
NDepend	Análisis avanzado de código .NET con métricas personalizables y tendencias históricas	C#, VB.NET
Checkstyle / PMD	Para Java: detectan violaciones de estilo, complejidad, duplicación, etc.	Java

Herramientas de Métricas de Calidad de Software

Métricas de Pruebas (Cobertura, eficacia, defectos):

Herramienta	Descripción
JaCoCo	Genera informes de cobertura de código para Java (líneas, ramas, métodos).
Istanbul / NYC (para Node.js)	Cobertura de código en JavaScript/TypeScript.
Coverage.py	Cobertura de pruebas en Python.
JUnit/TestNG + Reporters (Allure, ExtentReports)	Recopilan resultados de pruebas unitarias/integración y generan métricas (éxitos, fallos, tiempo).
Selenium + TestRail / Xray	Para pruebas funcionales/E2E: integran ejecución y reporte de defectos, cobertura de casos.

Herramientas de Métricas de Calidad de Software

Métricas de Proceso y Entrega (CI/CD, velocidad, estabilidad):

Herramienta	Descripción
Jenkins + plugins (e.g., Performance, Build Monitor)	Tiempo de build, frecuencia de fallos, duración, tasa de éxito.
GitLab CI / GitHub Actions	Métricas integradas: tiempo de pipeline, frecuencia de despliegues, rollback rate.
DORA Metrics Tools (a menudo combinadas con Prometheus + Grafana)	- Frecuencia de despliegue - Tiempo de entrega (lead time) - Tasa de fallos en producción - Tiempo medio de recuperación (MTTR)
Jira / Azure DevOps	Velocidad del equipo (story points), tasa de defectos abiertos/cerrados, ciclo de vida de incidencias.

Herramientas de Métricas de Calidad de Software

Métricas de Fiabilidad y Rendimiento en Producción:

Herramienta	Descripción
Prometheus + Grafana	Recopilan métricas de tiempo de actividad, latencia, errores (RED: Rate, Errors, Duration).
New Relic / Datadog / Dynatrace	APM (Application Performance Monitoring): errores por transacción, tracing distribuido, métricas de usuario real (RUM).
Sentry / Rollbar	Captura y análisis de errores en tiempo real: frecuencia, impacto, contexto.
ELK Stack (Elasticsearch, Logstash, Kibana)	Agregación y análisis de logs para detectar patrones de fallo o anomalías.

Herramientas de Métricas de Calidad de Software

Métricas de Experiencia del Usuario (UX/QoS):

Herramienta	Descripción
Google Lighthouse	Puntuación de rendimiento, accesibilidad, SEO, mejores prácticas, PWA.
Hotjar / Microsoft Clarity	Comportamiento del usuario (heatmaps, sesiones grabadas) → indirectamente relacionado con usabilidad y errores percibidos.
Apdex (índice de satisfacción del usuario)	Métrica derivada de tiempos de respuesta: Apdex = (Satisfechos + Tolerantes/2) / Total

Proceso de evaluación de métricas

01

Confiabilidad

Comprobación de la consistencia de los datos obtenidos mediante pruebas repetidas y técnicas de triangulación para validar resultados.

02

Pertinencia

Determinación de la relevancia de cada métrica al alinearla con los objetivos y requisitos específicos del proyecto de software.

03

Benchmarking

Comparación de las métricas con estándares conocidos de la industria para calificar el rendimiento relativo del software.

04

Análisis de tendencias

Identificación de patrones y variaciones en las métricas a largo plazo para prever problemas y optimizar el proceso.

Desafíos comunes en la recopilación de métricas

Datos inconsistentes

Resultados poco fiables debido a errores humanos o mal funcionamiento de las herramientas de medición deben ser abordados con revisiones adicionales.

Resistencia del equipo

La falta de cooperación por parte de los desarrolladores y testers puede ser mitigada mediante formación adecuada y concienciación sobre la importancia de las métricas.

Sobrecarga de datos

Exceso de información no procesable que dificulta la toma de decisiones requiere una simplificación y categorización cuidadosa de las métricas.

Falta de integración

Disparidad entre sistemas de recopilación y análisis de datos obliga a implementar softwares compatibles con procesos preexistentes.



03

Estándares de métricas: ISO/IEC 25022

Introducción a la norma ISO/IEC 25022

01

Objetivo principal

Proporcionar un marco estandarizado para evaluar la calidad del software.

02

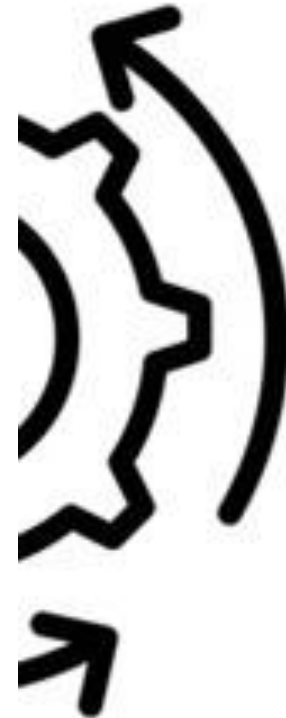
Aplicación clave

Facilitar la evaluación de las métricas de calidad del producto para su mejora continua.

03

Compatibilidad

Asegurar la uniformidad en las evaluaciones según estándares internacionales.



Implementación de la norma ISO/IEC 25022

Adaptación del sistema

Los proyectos deben alinear sus especificaciones de diseño y desarrollo con las métricas definidas en la norma para garantizar la calidad en el uso.



Evaluación continua

Proporciona herramientas y procesos para la supervisión constante de características clave como usabilidad y cobertura de contexto.



Análisis contextual

Los valores de las métricas deben ajustarse según el entorno específico del sistema y las necesidades del usuario, asegurando personalización y relevancia.



Beneficios del cumplimiento de la norma ISO/IEC 25022

Mejor experiencia del usuario

Al optimizar características de uso, los sistemas se vuelven más intuitivos y satisfactorios para los usuarios finales.

Calidad garantizada

Facilita un control riguroso de calidad en todas las etapas del ciclo de vida del software, promoviendo productos robustos y confiables.

Competitividad

Ayuda a las organizaciones a establecer credibilidad y diferenciación en mercados tecnológicos al seguir estándares internacionales reconocidos.

Reducción de errores

La adopción de métricas recomendadas minimiza defectos y fallos de usabilidad, mejorando la eficiencia operativa.

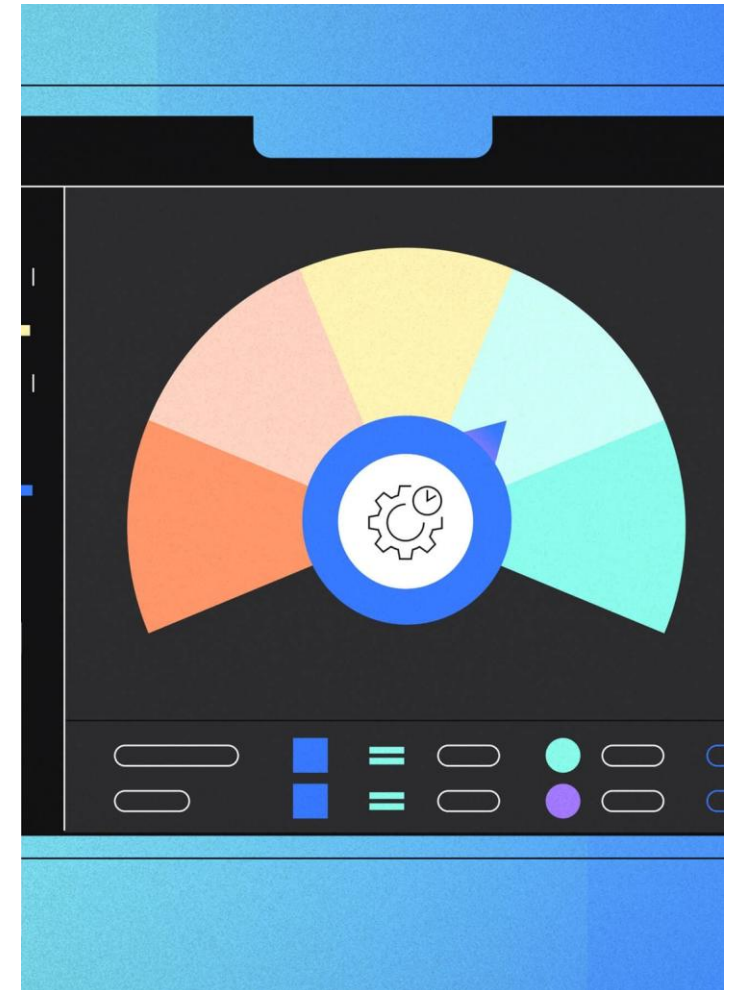


04

Métricas de procesos de software

Definición de Métricas de Proceso

- **Identificar el rendimiento**
Evaluar la eficiencia de los procesos para alcanzar objetivos definidos.
- **Mejorar la calidad del software**
Establecer indicadores para reducir defectos y aumentar la fiabilidad del producto.
- **Monitorear el progreso**
Controlar los tiempos y recursos utilizados en comparación con lo planificado.



Monitoreo del desempeño del desarrollo

1

Eficiencia del equipo

Seguimiento de métricas como el tiempo de vida de un pull request y la cantidad de pull requests fusionados para medir la productividad del equipo.

2

Optimización del flujo de trabajo

Identificar flujos o etapas ineficaces para mejorarlos, promoviendo una productividad constante y sostenida.

3

Productividad comparativa

Comparar métricas actuales con estándares previos o aceptados para establecer el progreso relativo del equipo.



Identificación de cuellos de botella en el proceso



Detección temprana

Medir variables como el tiempo de revisión de pull requests para descubrir procesos lentos que necesitan ajustes.

Soluciones específicas

Usar datos de métricas para diagnosticar directamente problemas estructurales o técnicos en el desarrollo.

Mejoras iterativas

Desarrollar estrategias basadas en métricas para abordar obstáculos, realizando cambios incrementales en las operaciones para optimizar el desarrollo.



05

Métricas del producto

Métricas de calidad funcional

01

Corrección

Grado en que el software cumple con sus especificaciones, solucionando los problemas previstos y logrando los objetivos del cliente con precisión.

02

Fiabilidad

Capacidad del software para realizar sus funciones esperadas sin fallos bajo condiciones establecidas, garantizando estabilidad operativa.

03

Usabilidad

Facilidad con la que los usuarios pueden aprender, utilizar y obtener resultados del software, reduciendo el tiempo de capacitación necesario.

04

Integridad

Nivel de protección contra accesos no autorizados al sistema y datos, asegurando la seguridad de la información sensible.

Métricas estructurales

Complejidad

Medición de la dificultad inherente al diseño y código del software, relacionada con la cantidad de dependencias y módulos.

01

02

Mantenibilidad

Esfuerzo necesario para localizar y corregir errores del programa en tiempo y forma eficiente.

Flexibilidad

Facilidad con la que se pueden realizar modificaciones en el software existente para adaptarlo a nuevas condiciones o requisitos.

03

04

Portabilidad

Capacidad del software para transferirse a diferentes entornos de hardware o sistemas operativos sin cambios significativos.

Medición de la satisfacción del usuario

01

Reusabilidad

Grado en que partes del software pueden emplearse para desarrollar otras aplicaciones, ofreciendo soluciones consistentes y modulares.

02

Interoperabilidad

Facilidad con la que el software puede interconectarse y funcionar en conjunto con otros sistemas o aplicaciones.

03

Experiencia del usuario

Evaluación de cómo el software cumple las expectativas y requisitos implícitos del cliente, garantizando una interacción agradable.

04

Adopción

Análisis del nivel de aceptación del software por parte de los usuarios finales basado en el tiempo de implementación y uso recurrente.



06

Métricas y prácticas ágiles

Tiempo de ciclo

Rapidez

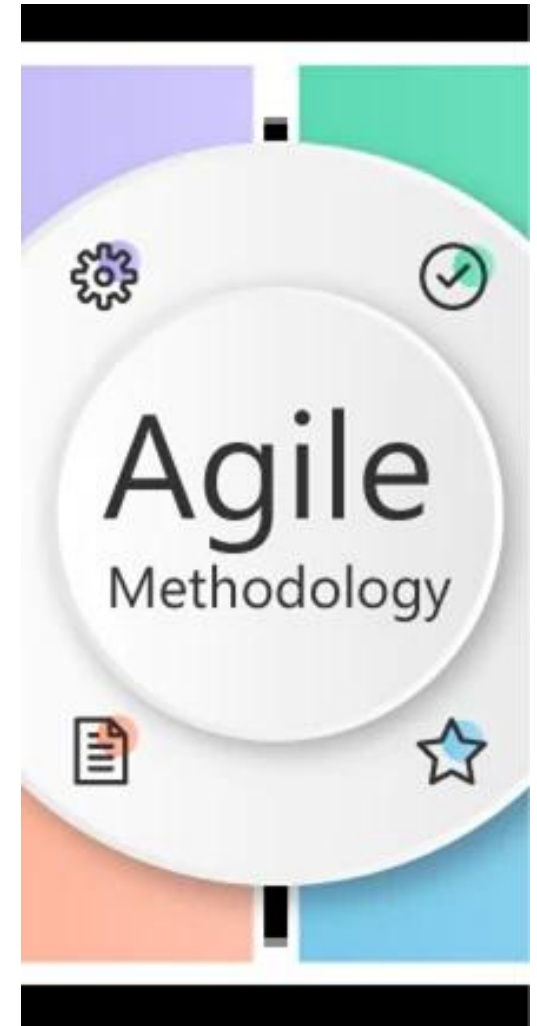
El tiempo de ciclo mide la rapidez con la que una tarea pasa desde su inicio hasta su finalización, permitiendo identificar ineficiencias.

Flujo continuo

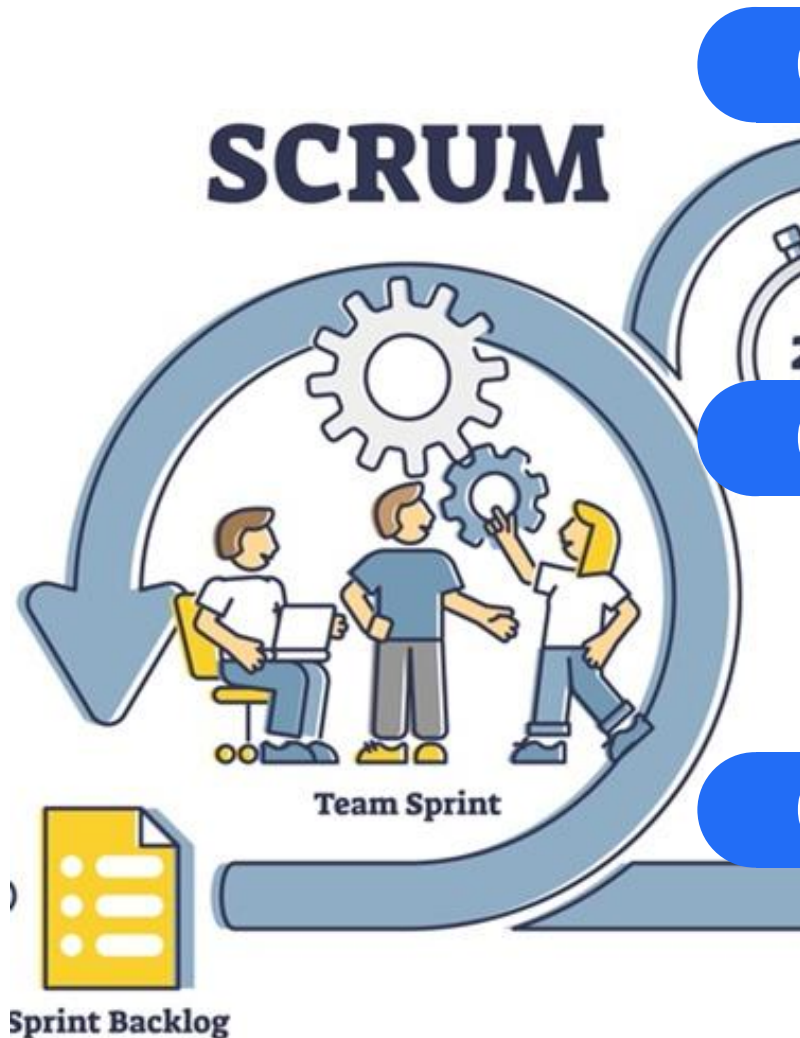
Un tiempo de ciclo más corto indica un flujo de trabajo más eficiente, lo cual es crucial para cumplir con los objetivos dentro de un marco ágil.

Identificación de bloqueos

Ayuda a localizar cuellos de botella en el proceso, permitiendo tomar medidas correctivas tempranas.



Velocidad del equipo



01

Productividad acumulativa

La velocidad refleja la suma de puntos de historias completadas en una iteración, proporcionando un indicador de la capacidad del equipo.

02

Planificación efectiva

Ayuda a establecer metas realistas para futuros sprints al basarse en datos históricos de rendimiento.

03

Consistencia

Monitorear la velocidad a lo largo del tiempo ayuda a evaluar la estabilidad y predictibilidad del equipo.

Gráficos de evolución

Transparencia

Los gráficos de burn-down y burn-up brindan una representación gráfica del trabajo completado y el trabajo restante.



Toma de decisiones

Facilita a los gerentes y partes interesadas evaluar si el proyecto marcha según lo planeado o si requiere ajustes.



Predicción

Mide el ritmo al que se resuelven tareas para prever con mayor precisión el tiempo requerido para completar proyectos futuros.





07

Desafíos en la implementación de métricas de software

Resistencia al cambio

Cultura organizacional

Falta de disposición de los equipos para modificar procesos existentes debido a la percepción de que las métricas añaden complejidad o limitan la autonomía.

Falta de capacitación

La ausencia de formación adecuada crea barreras para entender la relevancia de las métricas y cómo pueden mejorar los resultados del proyecto.

Interpretación errónea de las métricas

Trivialización

Usar métricas simples (como líneas de código) como indicadores completos puede llevar a decisiones basadas en datos incompletos o irrelevantes.



VS

Falta de contexto

Evaluar métricas en aislamiento, sin considerar el alcance o los objetivos del proyecto, puede resultar en una mala interpretación de su impacto real.

Gastos generales de métricas

● Sobrecarga administrativa

Monitorear demasiadas métricas puede consumir recursos y tiempo, desviando a los equipos de actividades clave del desarrollo.



● Falta de priorización

Medir aspectos irrelevantes puede generar pérdida de enfoque, dificultando la identificación de las prioridades críticas para el éxito del proyecto.

